Robust ML-Assisted Optimal Execution in Limit Order Books: Causal Microstructure Forecasting, Model Predictive Control, and Low-Latency Systems Evaluation

Othmane Hassani

***Abstract*—Optimal execution couples predictive signal, market mechanics, queue uncertainty, transaction costs, latency, and computation. We present a reproducible C++/Python research system that asks whether an improved causal limit-order-book forecast survives the downstream optimization map and improves execution. The design combines exact integer-arithmetic matching and accounting, explicit aggregate-L2 queue scenarios, strong classical baselines, matched non-ML and ML-assisted model-predictive controllers (MPC), a compact Conv1D–LSTM, imitation learning with dataset aggregation, categorical proximal policy optimization (PPO), registered stress tests, paired block-bootstrap inference, and end-to-end CPU/GPU measurement. All strategy findings are registered controlled-simulator evidence, not historical-market backtests; systems findings are direct measurements on identified hardware.**

The experiments reveal non-monotone proxy relationships. Calibration can improve log loss yet weaken controller response, and a perfect intermediate-event oracle can worsen implementation shortfall. An imitation student achieves exact controlled-holdout agreement but loses fidelity under distribution shift; PPO does not dominate strong deterministic heuristics. Across 43 stress conditions, the lowest-cost strategy changes in 16 of 42 non-central regimes. Of 129 paired contrasts, 85 confidence intervals cross zero, and 22 of 43 point winners have less than 80% bootstrap probability of remaining best. A profile-guided C++ optimization reaches 17.316 million operations/s in the measured four-thread workload. On a Tesla T4, transfer-inclusive GPU inference is slower for every registered batch-one workload; the temporal model becomes GPU-favorable at batch 256 with a 1.704× speedup. The evidence supports a methodological conclusion: predictive accuracy, decision value, policy complexity, robustness, and hardware acceleration should be evaluated through one causal execution-and-accounting pipeline rather than optimized as independent proxies.

***Index Terms*—optimal execution, limit order book, market microstructure, model predictive control, causal machine learning, imitation learning, reinforcement learning, robustness, reproducibility.**

O. Hassani is with ESILV Graduate School of Engineering, De Vinci Higher Education, Paris-La Défense, France. Source and artifacts: https://github.com/000zer000/robust-ml-optimal-execution.

I. Introduction

Electronic execution converts a parent trading instruction into a sequence of child orders while attempting to control explicit transaction costs, market impact, adverse price movement, timing risk, and non-completion risk. The classical optimal-execution literature formalizes this trade-off through dynamic control and market-impact models [1], [2], [3], [4], [5]. In modern limit-order-book (LOB) markets, however, the execution problem is more granular. The controller observes finite displayed depth, queue state only imperfectly, rapidly evolving order flow, fee and latency effects, and an action space containing both liquidity-providing and liquidity-taking decisions. The relevant state is therefore partly predictive and partly mechanical.

Machine learning offers a natural way to compress short-horizon order-flow information into forecasts. DeepLOB and subsequent work demonstrate that nonlinear models can extract predictive structure from LOB states and order-flow imbalance [6], [7], [8], [9]. Yet forecasting performance and execution value are not equivalent objectives. A model may reduce log loss without changing the optimizer's action; a poorly calibrated change near a decision boundary may alter an action more than a globally better predictor; and a target that is statistically learnable may be only indirectly related to implementation shortfall. This distinction is closely related to the predict-then-optimize and decision-focused learning literature [10], [11], [12].

A second complication is that policy sophistication is not synonymous with robustness. Reinforcement-learning (RL) execution systems can adapt to state-dependent environments [13], [14], [15], [16], but simulator misspecification, reward design, distribution shift, and random-seed sensitivity can dominate apparent improvements. A third complication is computational: inference and control latency become endogenous execution costs. A GPU kernel may be fast in isolation while host-device transfers make the complete decision path slower than CPU inference.

This work treats these issues as a single systems-and-decision problem. Rather than optimizing a forecasting benchmark, an RL score, or a microbenchmark independently, it builds a reproducible pipeline from market events to execution accounting and asks how each layer

changes downstream decisions. The research system is designed around five principles: exact market-state semantics, causal information timing, matched controller comparisons, stress testing under model uncertainty, and measurement-driven performance engineering.

A. Research Questions

The study is organized around the following questions.

RQ1: Can short-horizon causal microstructure forecasts alter execution decisions when injected into a queue-aware finite-horizon controller without changing the controller's optimizer or mechanical assumptions?

RQ2: Does improved predictive quality translate monotonically into improved decision quality or implementation shortfall?

RQ3: Can a compact learned policy reproduce an optimizer sufficiently well to reduce inference latency, and how does imitation degrade under covariate shift?

RQ4: Does an RL execution policy provide robust cost improvements over strong deterministic baselines across multiple seeds and stress regimes?

RQ5: How stable are strategy rankings under paired uncertainty analysis and multiplicity control?

RQ6: Which parts of the execution stack benefit from CPU parallelism, compiled inference, or GPU acceleration once end-to-end transfer costs are included?

B. Contributions

The primary contributions are:

- an exact, deterministic C++ market, queue-scenario, and execution-accounting core that gives classical schedules, MPC, imitation, and RL policies one mechanical comparison path;
- a matched forecast-to-control experiment combining a causal 20-feature model ladder with neutral, shuffled, stale, uncalibrated, perfect-event-oracle, and zero-weight ablations, isolating when predictive improvement does or does not create decision value;
- an integrated sequential-policy evaluation spanning exact-teacher imitation, validation-triggered DAgger, conservative OOD fallback, multi-seed categorical PPO, 43 registered stress conditions, and dependence-aware ranking inference;
- a reproducible systems study that measures correctness-preserving C++ optimization, multithread scaling, the Python/C++ boundary, compiled inference, and transfer-inclusive Tesla T4 timing on the complete registered workloads.

II. Related Work

A. Optimal Execution and Market Impact

The canonical execution problem trades immediacy against market impact and price risk. Bertsimas and Lo formulate execution as stochastic control [1]; Almgren and Chriss derive the widely used mean–variance frontier between expected impact cost and execution risk [2]. Subsequent models enrich impact dynamics and order-book structure, including transient resilience [3], general LOB shape functions [4], no-dynamic-arbitrage restrictions on impact kernels [5], and adaptive execution [17]. Limit and market order control introduces an additional fill-versus-adverse-selection trade-off [18], [19].

The present system uses these works as structural guidance rather than attempting to reproduce a single closed-form model. In particular, execution costs are computed from actual simulated fills and fees, while the controller includes explicit inventory and terminal penalties. This separation keeps accounting exact even when control heuristics are approximate.

B. Limit-Order-Book Dynamics and Queue Uncertainty

LOB state evolves through order arrivals, cancellations, modifications, and trades. Markovian and queue-reactive models show how state-dependent intensities and queue dynamics affect short-horizon price behavior [20], [21], [22]. Order-flow imbalance and book events are strongly connected to near-term price impact [23], [24]. Gould et al. provide a broad review of the empirical and modeling literature [25].

Aggregate L2 data creates a specific identification problem: a reduction in displayed size at a price cannot generally be decomposed uniquely into cancellations ahead of a child order, cancellations behind it, or hidden execution. The system therefore treats queue position as a scenario variable. The three queue models in Section IV-D deliberately bracket this uncertainty rather than turning aggregate depth into an unjustified exact fill probability.

C. Machine Learning for LOB Forecasting

DeepLOB combines convolutional and recurrent processing for LOB forecasting [6]; Sirignano and Cont study common price-formation structure across instruments [7]; and deep order-flow imbalance work demonstrates cross-horizon predictive content [8]. Recent surveys emphasize that target definition, normalization, leakage control, and horizon selection can be as important as architecture [9]. Calibration is also critical when predicted probabilities are consumed by downstream optimizers [26]. The gap addressed here is therefore not another forecasting leaderboard: it is an end-to-end, matched test of the path from forecast score to controller action, realized execution cost, stress robustness, and compute latency.

This work uses a deliberately compact temporal model and a strong simple-model ladder. The objective is not architecture novelty. Instead, forecasting is treated as an intermediate input whose value is tested by the execution controller.

D. Decision-Focused Prediction

Predict-then-optimize research demonstrates that minimizing prediction error need not minimize downstream decision loss [10]. Task-based and decision-focused methods

explicitly differentiate or otherwise learn through optimization objectives [11], [12]. The experiments in this paper do not replace the execution optimizer with a differentiable surrogate; instead, they perform controlled ablations that reveal where the prediction-to-decision map becomes non-monotonic. This provides a diagnostic foundation for future decision-focused training.

E. Imitation and Reinforcement Learning for Execution

DAgger addresses covariate shift in behavior cloning by querying an expert on learner-visited states and aggregating corrective data [27]. This mechanism is particularly relevant in execution because small action errors alter future inventory and market interaction, causing the state distribution to drift from the teacher's demonstration distribution.

RL has a longer history in execution, from early optimized trade-execution agents [13] to recent work on cross-environment generalization [14], continuous-time actor–critic methods [15], queue-reactive environments [28], and joint market/limit-order policies [16]. PPO provides a simple clipped policy-gradient objective with multiple minibatch epochs per on-policy sample [29]. The present experiments use one predeclared PPO family and report all registered seeds rather than selecting the best seed.

F. Market Simulation, Robustness, and Reproducibility

Simulation fidelity is central to any learning-based trading experiment. ABIDES provides an agent-based market simulation framework [30]; JAX-LOB demonstrates GPU-accelerated LOB simulation for large-scale RL [31]; realism metrics and benchmarking frameworks make simulator quality explicit [32], [33]; and recent work studies queue-reactive deep simulation and the simulation-to-reality gap [34], [35]. The statistical layer of this project draws on block-bootstrap and multiple-testing ideas appropriate for dependent experiments [36], [37], [38]. Reproducible configurations, seed reporting, compute disclosure, and deterministic regeneration follow the broader ML reproducibility principles summarized by Pineau et al. [39].

III. Problem Formulation

Consider a parent order of size $Q > 0$ submitted at decision time t_0, with side sign $s \in \{+1, -1\}$ for buy and sell, respectively. The benchmark price is the arrival mid or configured arrival benchmark P_0. A policy generates a sequence of child actions a_k at decision times t_k based on the causal state x_k. Fills are indexed by i and have quantity q_i, execution price P_i, and explicit fee/rebate contribution f_i.

A. Implementation Shortfall

Let executed notional and benchmark notional be

$$N_{\text{exec}} = \sum_i P_i q_i, \tag{1}$$

$$N_{\text{bench}} = P_0 Q, \tag{2}$$

with total fees $F = \sum_i f_i$. The signed implementation shortfall in quote units is

$$\text{IS} = s\,(N_{\text{exec}} - N_{\text{bench}}) + F, \tag{3}$$

and the normalized shortfall is

$$\text{IS}_{\text{bps}} = 10^4 \frac{\text{IS}}{N_{\text{bench}}}. \tag{4}$$

Positive values are worse execution; negative values represent price improvement relative to the arrival benchmark. The accounting layer computes Equation (3) from fills rather than from controller estimates. Incomplete orders do not receive a misleading final shortfall; terminal completion is charged explicitly through the configured completion mechanism.

B. State and Action Space

A generic execution state is

$$x_k = (B_k, A_k, I_k, \tau_k, z_k, \ell_k, c_k), \tag{5}$$

where B_k and A_k denote the observable bid/ask books, I_k is remaining inventory, τ_k is time remaining, z_k collects causal order-flow features, ℓ_k denotes latency state, and c_k contains fee/impact parameters. Classical policies use a subset of this state; learned policies receive explicitly frozen feature vectors.

The finite execution action family contains no action, passive placement at bounded fractions of remaining inventory, and aggressive execution. The RL environment expands this into six discrete actions:

$$\mathcal{A} = \{\text{wait}, P_{25}, P_{50}, A_{25}, A_{50}, A_{100}\}. \tag{6}$$

Action validation is centralized so that model, heuristic, and RL policies cannot bypass tick/lot, inventory, or terminal constraints.

C. Execution Cost Decomposition

For a child fill of fraction $\rho = q/Q$, the controlled environment uses a cost decomposition of the form

$$C_{\text{fill}} = C_{\text{price}} + \rho\,(C_{\text{impact}} + C_{\text{latency}} + C_{\text{fee}} + C_{\text{adverse}}), \tag{7}$$

where impact can scale as

$$C_{\text{impact}} = \eta \left(\frac{q}{D}\right)^{\gamma}, \tag{8}$$

Here D is visible depth, η is the impact coefficient, and the central controlled configuration sets $\gamma = 2$. Inventory exposure is penalized quadratically,

$$C_{\text{inv},k} = \lambda_I \left(\frac{I_k}{Q}\right)^2, \tag{9}$$

with an explicit terminal charge for forced completion.

IV. Exact Market and Execution Infrastructure

A. Canonical Event Representation

The simulator uses exact integer arithmetic for the market-state quantities that define matching semantics. Price is represented as signed 64-bit ticks, quantity as unsigned 64-bit lots, and quote notional as signed integer quote atoms with rational tick and lot sizes. Timestamps are nanosecond-resolution integers accompanied by an explicit clock domain. This eliminates floating-point ambiguity from price-level identity and quantity conservation.

Deterministic event ordering uses the lexicographic key

$$K(e) = (t_e,\, s_e,\, u_e,\, g_e,\, i_e), \tag{10}$$

where t_e is event time, s_e canonical sequence, u_e source subsequence, g_e ingestion sequence, and i_e internal event identifier. Canonical JSONL audit events are chained by SHA-256, allowing replay artifacts to be checked for accidental mutation.

B. Matching Engine

The C++ matching engine enforces price-time ordering at the modeled level, exact tick/lot validation, quantity conservation, deterministic child-order identifiers, and explicit order lifecycle state. Aggressive execution sweeps visible depth level by level; passive execution is delegated to the queue model and cannot fill before activation. The same accounting objects feed execution metrics, controller tests, and policy environments.

C. Data Admission and Replay

Historical/live ingestion is organized around immutable source manifests, clock metadata, symbol normalization, schema validation, and quarantine rules. Sequence gaps, malformed timestamps, impossible book states, and incomplete spans are rejected rather than silently interpolated. The controlled numerical experiments reported in this manuscript are generated by the validated deterministic simulation path; hardware measurements are empirical on the processors identified in Section XIII. Keeping these evidence classes separate makes model-risk experiments reproducible while preserving a distinct external-feed replay path.

D. Aggregate-L2 Queue Models

At passive activation, a child order is placed behind all displayed same-price quantity. Subsequent increases at that level are placed behind the child. The ambiguous case is an unexplained reduction in displayed size. Three queue assumptions bracket the unknown position of cancellations:

1) **Optimistic:** unexplained displayed reductions are allocated ahead of the child, reducing queue-ahead quantity.
2) **Central:** unexplained reductions are allocated proportionally between quantity ahead of and behind the child.
3) **Pessimistic:** unexplained reductions are allocated behind the child; only eligible trades reduce queue ahead.

The queue state is therefore a controlled uncertainty model rather than an exact inferred FIFO position. This design is consistent with the identification limits of aggregate L2 data and provides a direct robustness axis.

TABLE I
Evidence Tiers and Permitted Interpretation

Tier	Evidence	Permitted claim
Controlled	registered simulator episodes	comparison within generator, controller, and stress assumptions
Systems	identified CPU/T4 measurements	workload- and hardware-specific performance
Historical	contracts only; test unopened	engineering readiness, not strategy performance

V. Experimental Design and Evaluation Protocol

The experimental design separates forecasting quality, controller behavior, sequential policy performance, robustness, statistics, and systems timing. This separation is intentional: a change in an upstream metric is not interpreted as economically valuable unless the downstream layer is evaluated directly.

A. Evidence Tiers and Claim Boundary

The evidence is divided before analysis into three tiers. Strategy conclusions use registered deterministic simulator populations so mechanisms can be isolated under identical random seeds. Systems conclusions use direct measurements on the CPU and Tesla T4 environments recorded with the artifacts. Historical-feed ingestion, sequence admission, and replay interfaces are implemented, but the locked historical strategy test remains unopened because no qualifying dataset has been admitted. Consequently, the paper makes no historical profitability or deployment-performance claim.

B. Controlled Populations and Split Discipline

The microstructure-prediction experiment uses 100 chronological controlled day identifiers spanning BTCUSDT and ETHUSDT and both passive sides. The temporal sequence corpus contains 2,000 eight-step examples partitioned into 1,000 training, 400 validation, 200 calibration, and 400 untouched holdout sequences. Each candidate horizon uses the same split roles. Training fits weights and normalizers; validation chooses hyperparameters and stopping epoch; calibration estimates probability calibration; the holdout is evaluation-only.

Sequential execution experiments use independent episode populations rather than reusing the forecasting

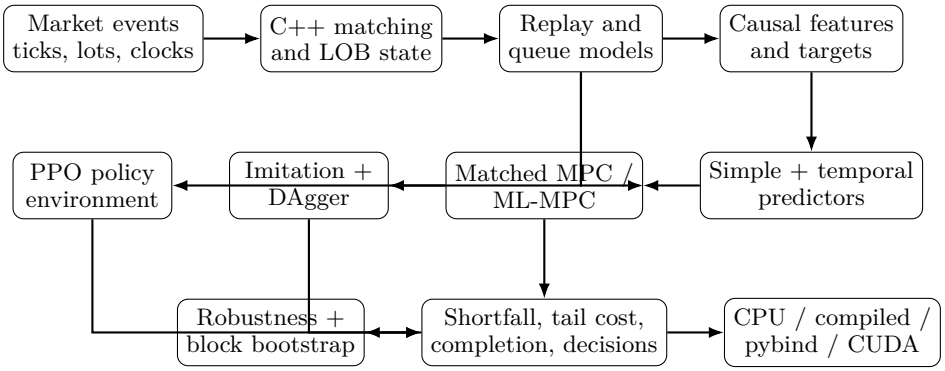


Fig. 1. Research-system architecture. Market mechanics, causal prediction, decision policies, statistical analysis, and systems performance share the same execution-accounting layer.

TABLE II
Primary Experimental Populations

Stage	Population	Role
Temporal ML	1000/400/200/400 seq.	train/val/cal/holdout
Imitation	80/30/30/30/30 ep.	train/val/correct/holdout/OOD
PPO	5 seeds	aggregate all seeds
Robustness	43 × 24 paired ep.	stress evaluation
Statistics	129 contrasts	block bootstrap

holdout. Imitation learning uses 80 training episodes, 30 validation episodes, 30 correction-pool episodes, 30 controlled-holdout episodes, and 30 OOD episodes, with six decisions per episode before environment-dependent termination. The RL experiment trains across three controlled regime families and evaluates on distinct ID and OOD regime sets. The Step-28 robustness matrix then reuses a fixed set of 24 paired episode seeds inside each stress cell so every policy experiences the same stochastic environment realization.

C. Forecast Metrics

For binary target y_i and probability p_i, log loss and Brier score are

$$\mathcal{L}_{\log} = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log p_i + (1-y_i)\log(1-p_i)\right], \quad (11)$$

$$\mathcal{L}_{\text{Brier}} = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)^2. \quad (12)$$

Expected calibration error partitions predictions into bins $\mathcal{B}_m$ and computes

$$\text{ECE} = \sum_m \frac{|\mathcal{B}_m|}{n}\left|\text{acc}(\mathcal{B}_m) - \text{conf}(\mathcal{B}_m)\right|. \quad (13)$$

ROC-AUC and PR-AUC are retained because class prevalence differs sharply by horizon, but probability-sensitive metrics are central because the downstream MPC consumes probabilities rather than thresholded labels.

For the linear baseline,

$$p(y=1 \mid x) = \sigma(w^\top x + b), \quad (14)$$

and Platt calibration maps an uncalibrated logit z to

$$p_{\text{cal}} = \sigma(az + b_c), \quad (15)$$

with a, b_c fitted only on the calibration partition.

D. Execution and Policy Metrics

The primary economic measure is implementation shortfall in Equation (4). Sequential evaluations additionally report completion rate, invalid-action rate, residual inventory, action frequencies, mean cost, p95 cost, and $\text{CVaR}_{0.95}$. For episode costs $C_1, \ldots, C_n$, empirical tail cost is

$$\widehat{\text{CVaR}}_{0.95} = \frac{1}{|\mathcal{T}|}\sum_{i\in\mathcal{T}} C_i, \quad (16)$$

where $\mathcal{T}$ contains observations at or above the empirical 95th-percentile threshold. Imitation adds teacher-action agreement and fallback rate. Prediction-to-decision analysis reports whether a probability improvement changes the MPC action and whether the resulting execution cost changes.

E. Selection and Ablation Discipline

Model-family comparisons, calibration, controller sensitivity, imitation correction, and RL evaluation are separated by fixed data roles and configuration files. Neutral/base-rate controls test whether the ML controller collapses to the non-ML path; shuffled and stale predictions test dependence on signal information; a perfect intermediate-event oracle tests target-objective alignment; zero prediction weight tests solver equivalence. PPO reports all five registered seeds and the robustness matrix uses paired episode seeds. This design reduces the opportunity for post-hoc winner selection and makes negative or invariant results visible in the same artifacts as positive results.

VI. Classical Execution Policies and Receding-Horizon MPC

A. Baselines

The baseline family contains immediate execution, a time-sliced TWAP-like schedule, a liquidity-aware policy,

and an Almgren–Chriss-style control benchmark. Immediate execution establishes the cost of maximal urgency. TWAP-like execution distributes quantity over the horizon. The liquidity-aware policy conditions aggressiveness on visible spread/depth and remaining inventory. These baselines are intentionally strong enough that an RL or ML controller must improve a meaningful comparator rather than a trivial policy.

B. Queue-Aware Local Fill Proxy

The non-ML MPC uses a bounded local passive-fill proxy to rank candidate action sequences inside a short solve. Let Q_s and Q_o denote same- and opposite-side top-level depth and $F \in [0,1]$ a normalized flow signal. The queue-share term is

$$q_{\text{share}} = \frac{Q_s}{Q_s + Q_o}, \tag{17}$$

and the local proxy is

$$\hat{p}_{\text{fill}} = \text{clip}\left(p_0 + w_q(0.5 - q_{\text{share}}) + w_f(F - 0.5), 0, 1\right). \tag{18}$$

This quantity is a controller heuristic, not a learned probability model. It is held fixed within a local search and recomputed after the environment advances.

C. Finite-Horizon Search

The MPC enumerates a bounded action tree with planning horizon at most four, action fractions $\{1/4, 1/2, 1\}$, and maximum passive fraction $1/2$. Only the first action of the lowest-cost plan is executed; the state is then observed again and the optimization repeats. This receding construction limits open-loop sensitivity.

For a candidate stage k, a generic cost is

$$J_k = C_{\text{exec},k} + \lambda_I \left(\frac{I_{k+1}}{Q}\right)^2 + J_{k+1}, \tag{19}$$

where an aggressive branch uses visible-sweep cost plus fees and depth penalties, while a passive branch uses expected attempted fill under Equation (18). The terminal cost is

$$J_T = \frac{I_T}{Q}\left(C_{\text{forced}} + \lambda_T\right) + \lambda_Q \left(\frac{I_T}{Q}\right)^2. \tag{20}$$

The central controller parameters use $\lambda_I = 2\,\text{bps}$, $\lambda_T = 20\,\text{bps}$, and $\lambda_Q = 10\,\text{bps}$. These terms do not replace exact accounting; they only shape the optimizer's action choice.

VII. Causal Microstructure Prediction

A. Target Definition

For each candidate horizon $h \in \{250\text{ ms}, 1\text{ s}, 5\text{ s}\}$, the primary binary target is same-side best-quote depletion or trade-through within h. If the passive side at decision time is $d \in \{\text{bid}, \text{ask}\}$, the target can be written

$$y_h(t) = \mathbf{1}\{\text{same-side best quote depletes or is traded through in } (t, t+h]\}. \tag{21}$$

Input: causal observation x_k, remaining inventory I_k
Build bounded candidate action tree $\mathcal{T}(x_k, I_k)$
Evaluate passive branches using queue state and local fill proxy
Evaluate aggressive branches by visible sweep and fee/impact terms
Add inventory-risk and terminal-completion costs
$a_k \leftarrow$ first action of lowest-cost path
Validate a_k against inventory and market constraints
Execute a_k, advance environment, observe x_{k+1}
Repeat until $I_k = 0$ or terminal completion
Algorithm 1: Receding-Horizon Execution Control

A side-signed adverse-move target is retained as a secondary diagnostic. The primary target is deliberately described as a quote-depletion/trade-through event; it is not algebraically identified with the realized fill of a specific child order.

B. Causal Information Contract

Let a decision occur at wall-clock time t and let the market observation cutoff be

$$c = t - \ell_{\text{obs}}, \tag{22}$$

where ℓ_{obs} is observation latency. A source event e can contribute to the feature vector only if

$$t_e \le c \quad \text{and} \quad t_e^{\text{available}} \le t. \tag{23}$$

This dual-time rule prevents both future-event leakage and availability-time leakage. Mutation tests perturb events after the cutoff and require the frozen feature vector to remain unchanged.

C. Frozen Feature Set

The feature contract contains 20 causal variables, grouped as follows:

- **spread/depth:** spread in ticks; same/opposite top-1 lots; same/opposite top-5 lots;
- **imbalance:** side-aligned top-1 and top-5 imbalance in basis-point-scaled units;
- **trade flow:** toward-quote signed trade flow over 250 ms, 1 s, and 5 s; trade counts over 1 s and 5 s;
- **price dynamics:** side-aligned mid moves over 250 ms, 1 s, and 5 s; realized absolute mid movement over 1 s and 5 s; 1 s spread change;
- **staleness:** quote age and time since last trade.

Exact names and units are reproduced in Appendix A.

D. Simple Model Ladder and Calibration

Each horizon is first modeled by a deliberately compact ladder: training base rate, L2-regularized logistic regression, histogram gradient-boosted trees, and a one-hidden-layer MLP. Logistic and MLP inputs use a scaler fitted on

TABLE III
TEMPORAL MODEL CONFIGURATIONS

Horizon	Conv	LSTM	Params.	Epoch
250 ms	12	12	1,993	3
1 s	8	8	1,073	4
5 s	12	12	1,993	5

training data only. Hyperparameters are selected on validation data; Platt scaling is fitted on a separate calibration partition. Metrics include log loss, Brier score, expected calibration error, calibration slope/intercept, ROC-AUC, PR-AUC, and temporal/instrument/passive-side slices.

E. Temporal Model

The temporal predictor consumes an eight-step causal history $X \in \mathbb{R}^{8\times 20}$. The architecture is

$$X \xrightarrow{\text{causal Conv1D}(k=3)} H_c \xrightarrow{\text{GELU}} H_g \xrightarrow{\text{LSTM}} h_8 \xrightarrow{\text{linear}} z, \tag{24}$$

followed by probability $\sigma(z)$ and optional Platt transform $\sigma(az+b)$. Left padding preserves causality. The controlled dataset contains 4,800 source rows and 2,000 sequences split into 1,000 training, 400 validation, 200 calibration, and 400 controlled-holdout sequences. The scaler is fitted on training only; early stopping uses validation only; calibration uses calibration only.

All configurations use AdamW, learning rate 5×10^{-3}, weight decay 10^{-4}, gradient clipping, deterministic CPU training, no dropout, maximum 14 epochs, and patience 3. Training prevalences are 4.5%, 17.8%, and 58.9% for 250 ms, 1 s, and 5 s respectively.

VIII. ML-Assisted MPC and Prediction-to-Decision Analysis

A. Matched Controller Construction

The ML-assisted controller reuses the exact same finite-horizon search, action space, queue model, latency model, aggressive-cost implementation, inventory penalty, and terminal handling as the non-ML MPC. The intended difference is a prediction-derived passive-risk adjustment. For predicted event probability p and training base rate $\bar{p}$, the additive adjustment is

$$\Delta C_{\text{pred}} = \lambda_p (p - \bar{p}), \tag{25}$$

where λ_p is a controlled sensitivity weight in basis points. Centering at the base rate creates an exact neutral control: replacing every prediction by $\bar{p}$ produces zero adjustment at every decision. Setting $\lambda_p = 0$ separately verifies that the ML path reduces byte-for-byte to the non-ML decision accounting in the deterministic fixture.

Every prediction endpoint carries decision identifier, endpoint time, feature cutoff, availability time, horizon, model identifier, provenance identifier, probability, base rate, and prediction kind. The controller rejects duplicate, unavailable, future, or mismatched endpoints rather than substituting a default probability.

TABLE IV
CONTROLLED-HOLDOUT LOG LOSS AND CONTROLLER SENSITIVITY

Horizon	Base	Raw	Cal.	First change†
250 ms	0.1453	**0.1442**	0.1618	5,000 / none
1 s	0.4829	**0.4738**	0.4780	none / none
5 s	0.6895	0.6675	**0.6401**	10,000 / 5,000

†calibrated / uncalibrated prediction-weight threshold in bps.

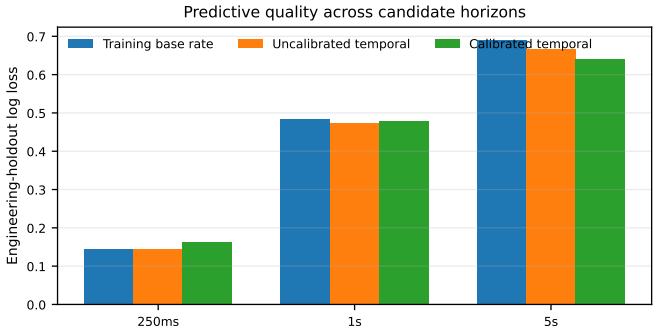


Fig. 2. Predictive quality across candidate horizons on the 400-sequence controlled holdout per horizon. Lower log loss is better.

B. Ablation Design

The registered inputs are calibrated model probability, uncalibrated model probability, training base rate, within-day/instrument shuffled predictions, stale predictions, a perfect intermediate-event oracle, and zero prediction weight. Decision sensitivity is evaluated on

$$\lambda_p \in \{0, 50, 100, 250, 500, 1000, 2000, 5000, 10000, 25000\} \text{ bps}. \tag{26}$$

This separates three questions: whether a predictor is statistically better, whether its output changes the controller, and whether the changed action improves downstream execution cost.

C. Predictive Results

Table IV and Figure 2 show a non-monotonic mapping. At 5 s, calibration improves log loss from 0.6675 to 0.6401, yet the uncalibrated signal changes the controller at a lower sensitivity weight. At 250 ms, calibration worsens log loss but can still cross a decision boundary at high weight while the raw signal does not. At 1 s, both learned variants improve on the base-rate log loss, but neither changes the controller anywhere on the registered grid.

Across the registered comparison-weight cases, prediction and decision outcomes decompose into 84 cases in which prediction improves but the decision is unchanged, 26 in which both improve/change, 32 in which prediction does not improve and the decision is unchanged, and 8 in which prediction does not improve yet the decision changes. Figure 3 visualizes this separation.

A particularly useful diagnostic is the perfect event oracle. It predicts the intermediate quote-depletion/trade-through target without classification error, yet can still

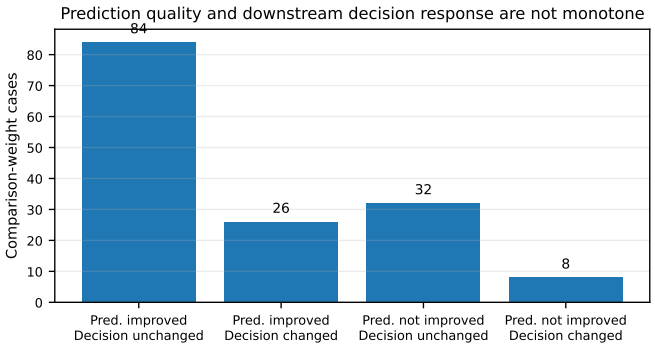


Fig. 3. Relationship between predictive improvement and downstream action response. The optimizer often absorbs prediction changes without changing its first action.

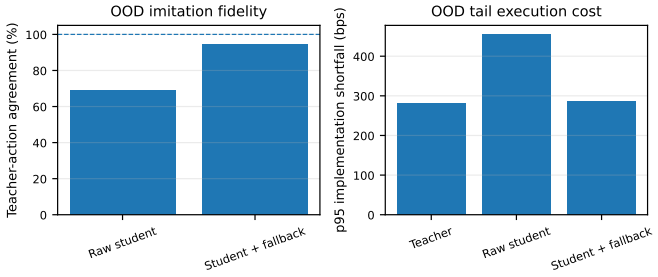


Fig. 4. OOD imitation behavior. A validation-selected teacher fallback recovers most action fidelity and tail-cost behavior, but at a high fallback rate.

worsen implementation shortfall in the controlled execution fixture. This demonstrates that an oracle for an intermediate microstructure target is not an oracle for the execution objective. The result motivates explicit decision-value evaluation and, potentially, future decision-focused training.

IX. Imitation Learning for Low-Latency Policy Approximation

A. Teacher Construction

A first behavior-cloning attempt exposed a degenerate teacher action surface: under the conservative non-ML MPC configuration, nearly all sampled controlled states selected the same first action. Rather than accepting a misleadingly high imitation accuracy, the teacher-data generator was expanded over controlled state dimensions and uses the already validated shared MPC optimizer with a causal controlled prediction-risk input to expose multiple optimizer action classes.

The training teacher distribution contains 480 labeled states with class counts 297 passive-50, 162 aggressive-100, 19 aggressive-50, and 2 aggressive-25. The C++ teacher oracle returns both the exact MPC action and adaptive signal vector for arbitrary causal states, which allows expert relabeling on learner-visited states.

B. Behavior Cloning and DAgger

The student is a one-hidden-layer MLP. Validation-only model selection chooses eight hidden units and L2 regularization $\alpha = 10^{-4}$. Let π_T be the teacher and π_θ the student. Initial behavior cloning minimizes categorical cross-entropy

$$\mathcal{L}_{\mathrm{BC}}(\theta) = -\sum_{(x,a)\in\mathcal{D}} \log \pi_\theta(a \mid x). \tag{27}$$

A predeclared sequential validation agreement threshold of 98% determines whether correction is required. The initial student achieves 97.70%, triggering one DAgger-style correction round. Learner rollouts over a dedicated correction pool generate visited states $x \sim d_{\pi_\theta}$, the C++ oracle labels $a = \pi_T(x)$, and 87 corrective examples are appended before retraining.

After correction, validation agreement increases to 98.84%. On the untouched controlled holdout, the student reproduces the teacher action path exactly, with 100% action agreement, 100% completion, zero invalid actions, and zero shortfall delta relative to the teacher.

C. Out-of-Distribution Behavior and Fallback

OOD regimes intentionally shift liquidity, volatility, latency, and execution parameters. Raw student agreement falls to 69.30%, and p95 implementation shortfall increases to 455 bpsversus 281.45 bpsfor the teacher. A validation-selected conservative fallback sends uncertain states back to the teacher and raises agreement to 94.78%, with p95 shortfall 287.5 bps. The fallback is invoked on 82.61% of OOD decisions, exposing the latency-versus-robustness trade-off directly.

X. Reinforcement Learning Environment and PPO

A. Markov Decision Process

The controlled execution environment exposes an 11-dimensional causal state: remaining-inventory fraction, time remaining, scaled spread, depth ratio, imbalance, volatility, latency, fee, impact coefficient, recent fill, and adverse momentum. The action set is given by Equation (6). Episodes contain 12 decision steps and a parent size of 100 lots in the central configuration. Action masks prevent terminal or inventory-infeasible decisions.

The environment is interactive rather than a static classifier: aggressive participation induces temporary impact on subsequent state transitions. Training regimes cover normal, active, and thin liquidity. OOD evaluation includes thin/high-volatility, high-latency, adverse-fee, high-impact, and combined regimes.

B. Reward Accounting

For each transition, the environment stores raw fill price, fill quantity, visible depth, arrival price, fee, latency,

TABLE V
CONTROLLED RL EVALUATION (LOWER COST IS BETTER)

Policy	ID μ	ID C95	OOD μ	OOD C95
PPO (5 seeds)	1.214	11.942	7.333	26.961
Liquidity-aware	**0.820**	11.026	8.554	23.559
TWAP-like	2.488	7.272	**7.004**	**17.821**
Immediate	3.879	**6.649**	13.442	33.069

impact, adverse-momentum input, inventory before action, and terminal-fill information. The reward is

$$r_k = -\left(C_{\text{fill},k} + \lambda_I \left(\frac{I_{k+1}}{Q} \right)^2 + C_{\text{terminal},k} \right). \tag{28}$$

A separate reconstruction function recomputes the economic cost from the raw transition facts. The deterministic reward audit obtains absolute reconstruction error 0.0. Additional tests verify that terminal inventory cannot disappear, executed quantity cannot exceed the parent order, invalid actions are masked consistently, and future random draws cannot alter the current observation.

C. PPO Objective and Configuration

The policy uses two 32-unit tanh hidden layers with a categorical action head and value head. With probability ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}, \tag{29}$$

PPO optimizes the clipped objective [29]

$$L^{\text{clip}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right]. \tag{30}$$

The registered configuration uses $\gamma = 0.99$, GAE $\lambda = 0.95$, clipping $\epsilon = 0.2$, entropy coefficient 0.01, value coefficient 0.5, learning rate 10^{-3}, 20 training episodes per update, four update epochs, and ten updates. Five independent seeds $\{27, 127, 227, 327, 427\}$ are all retained.

D. Multi-Seed Results

The PPO aggregate achieves 100% completion and zero invalid-action rate across registered ID and OOD evaluation episodes, but it is not uniformly lowest cost. Liquidity-aware execution has lower ID mean cost, while TWAP-like execution has lower OOD mean and OOD CVaR95. PPO seed variation is material: ID mean cost ranges approximately from 0.20 to 3.34 bpsacross the five registered seeds. This is precisely why the aggregate and all seeds are retained rather than reporting a best run.

XI. ROBUSTNESS MATRIX

A robust execution policy should not be selected from a single central configuration. The robustness experiment therefore evaluates 43 registered stress cells, each using the same 24 paired episode seeds for all comparable policies. Dimensions include observation latency, decision-grid resolution, visible liquidity, spread, volatility, aggregate-L2 queue assumptions, fees/rebates, parent-order size, horizon, impact magnitude and functional form, prediction degradation, data drop/delay, temporal/instrument distribution shift, compute constraints, and simulator mismatch.

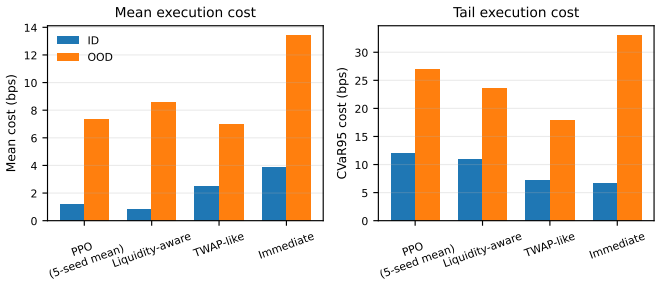


Fig. 5. Mean and tail execution cost for PPO and strong deterministic baselines in controlled ID/OOD evaluation.

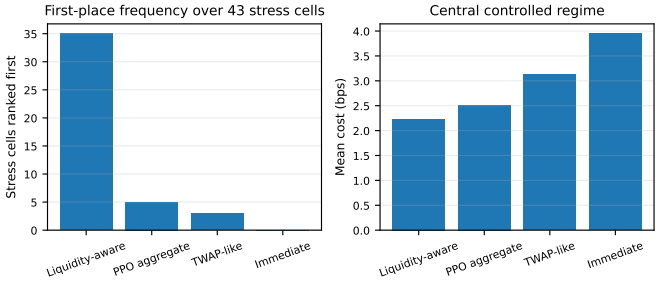


Fig. 6. Robustness summary: first-place frequency across 43 stress cells and central-regime mean execution cost.

A. Ranking Changes

The central regime mean costs are 2.232 bpsfor liquidity-aware, 2.514 bpsfor PPO aggregate, 3.123 bpsfor TWAP-like, and 3.947 bpsfor immediate execution. Across all 43 cells, liquidity-aware ranks first in 35, PPO aggregate in 5, TWAP-like in 3, and immediate in none. More importantly, the identity or ordering of strategies changes in 16 of the 42 non-central cases.

The stress matrix reveals qualitatively different failure modes. Under a volatility shock, mean costs are approximately 0.824 bps(liquidity-aware), 3.202 bps(TWAP-like), 4.397 bps(immediate), and 6.988 bps(PPO). Under the combined unseen regime they become 12.182, 13.363, 32.511, and 18.044 bpsrespectively. In one adverse simulator-mismatch cell, however, PPO becomes the point winner at 4.667 bps, narrowly ahead of TWAP-like at 4.737 bpsand liquidity-aware at 5.913 bps. Thus robustness is regime-dependent rather than reducible to a single policy ranking.

Worst mean-cost increases relative to the central regime are approximately +28.56 bpsfor immediate, +15.53 bpsfor PPO aggregate, +10.24 bpsfor TWAP-like, and +9.95 bpsfor liquidity-aware. These values quantify sensitivity rather than merely counting wins.

XII. Dependence-Aware Statistical Analysis

A. Paired Contrasts

Every comparable stress cell uses paired episode seeds, so inference is performed on paired cost differences rather than independent policy samples. If $C_{i,A}$ and $C_{i,B}$ are costs for paired episode i, the effect variable is

$$d_i = C_{i,A} - C_{i,B}. \tag{31}$$

Positive values imply higher cost for A.

B. Moving-Block Bootstrap

To avoid treating ordered pseudo-day observations as IID, the controlled analysis uses a moving-block bootstrap. A frozen autocorrelation rule selects block length $L = 5$: the relevant paired-difference autocorrelation remains below $|\rho| < 0.1$ for two consecutive lags beginning at lags 5 and 6. Each registered contrast uses $B = 4096$ bootstrap replications. A bootstrap sample concatenates contiguous blocks while preserving the within-episode policy pairing.

For statistic $\hat{\theta}$, the empirical percentile interval is

$$\left[\hat{\theta}^{*(\alpha/2)}, \hat{\theta}^{*(1-\alpha/2)}\right], \qquad \alpha = 0.05. \tag{32}$$

Exploratory stress families use Holm step-down multiplicity adjustment [38]. In addition to pairwise effects, a ranking bootstrap resamples the common paired observations and recomputes the lowest-cost policy, producing a probability that the point-estimate winner remains first.

C. Uncertainty Results

There are 129 paired controlled contrasts. Of these, 85 have 95% intervals crossing zero. Of the 43 stress-cell point winners, 21 achieve at least 80% bootstrap probability of remaining best and 22 do not. The central point winner, liquidity-aware, has bootstrap win probability 87.84%; PPO aggregate has 8.13%, TWAP-like 2.22%, and immediate 1.81%.

For the central PPO-minus-liquidity-aware contrast, the mean paired difference is approximately $+0.282$ bpswith 95% interval $[-0.106, 0.658]$ bps. The point ranking therefore contains substantial uncertainty even though the mean costs differ. Several apparent PPO/TWAP point wins are more fragile: the 25%-depth PPO winner retains only approximately 39.5% bootstrap win probability, while narrow-spread TWAP and selected simulator-mismatch PPO wins remain below 50%.

This analysis changes the interpretation of the robustness table: first-place counts summarize observed point estimates, while the bootstrap quantifies how often those rankings survive paired resampling.

XIII. High-Performance Systems Evaluation

Execution logic operates under tight decision budgets, so compute latency is treated as an execution-system variable. The performance study follows a profile-first workflow: measure a representative workload, identify a concrete bottleneck, apply one narrow optimization, rerun correctness, and then benchmark scaling.

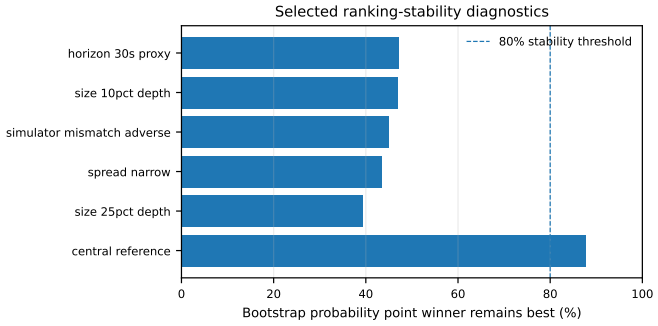


Fig. 7. Bootstrap probability that selected stress-cell point winners remain lowest cost. The dashed line marks the preregistered 80% ranking-stability diagnostic.

TABLE VI
C++ Matching Benchmark

Threads	Base ms	Opt. ms	Speedup	Opt. Mops/s
1	13.527	13.286	1.018×	4.516
2	6.895	6.591	1.046×	9.103
4	4.932	3.465	1.423×	17.316

A. C++ Matching Engine

The microbenchmark repeatedly rests and immediately matches orders through the real C++ matching engine, with 60,000 submit operations per repetition. Profiling identifies measurable unordered-map growth and rehashing overhead. The optimization adds an optional expected-order-count capacity hint that reserves identifier/history tables in advance. The default remains zero, preserving prior allocation behavior. A direct regression requires the hinted and unhinted engines to produce identical canonical book state and deterministic checksum.

The process-level 100k-pair memory check is effectively flat: measured maximum resident set size is approximately 49,544 KiB before and 49,128 KiB with the capacity hint. The optimization therefore targets rehash work without introducing a material memory expansion in the measured workload.

B. CPU Neural Inference

The primary CPU benchmark host exposes five AMD EPYC 9V74 logical/physical CPUs, approximately 6.37 GB memory, GCC 14.2, Python 3.13.5, NumPy 2.3.5, scikit-learn 1.8.0, and PyTorch 2.10.0+cpu. Batch-one and batched inference are measured separately because low-latency decisions and offline throughput are different workloads.

For the imitation policy, the measured batch-one median is 11.88 μs in NumPy and 9.84 μs through the traced PyTorch compatibility path; at batch 256 the traced path reaches approximately 9.54 million rows/s. For PPO, batch-one eager inference is approximately 31.15 μs in the primary CPU benchmark and the traced path approximately 15.84 μs; at batch 256 it reaches approximately 3.13 million rows/s. The temporal model benefits strongly

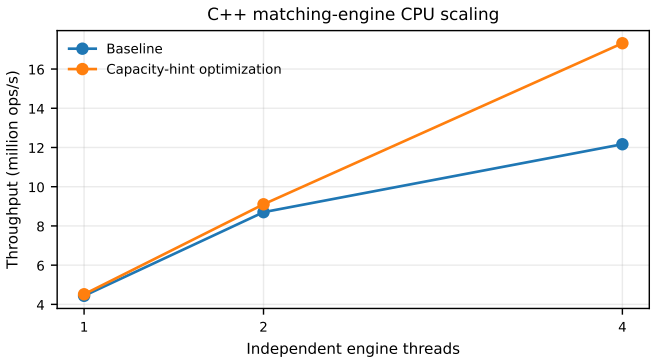


Fig. 8. Measured C++ matching throughput for independent engines as thread count increases.

TABLE VII
TESLA T4 TRANSFER-INCLUSIVE INFERENCE

Workload	Batch	CPU μs	T4 μs
Imitation	1	28.68	181.70
PPO	1	69.96	341.24
Temporal 5 s	1	485.85	662.06
Temporal 5 s	32	682.17	713.58
Temporal 5 s	256	1281.69	**752.35**

from batching, reaching hundreds of thousands of rows/s depending on thread count.

Current PyTorch compiler paths are measured separately from deprecated compatibility paths. The temporal LSTM can be captured with `torch.export`; full-graph `torch.compile` is unsupported for the measured recurrent path, while graph-break compilation is slower at batch one. This distinction prevents a fast legacy trace measurement from being mistaken for a future deployment recommendation.

C. Python–C++ Boundary

The existing binding is built against pybind11-compatible headers and verified by an exact diagnostic-sequence equality check before timing. For the small diagnostic call, the C++ path through pybind has median 304.05 ns (p95 311.97 ns), compared with 1,748.6 ns (p95 1,761.3 ns) for the pure-Python reference. This microbenchmark isolates call-boundary overhead; it is not interpreted as an end-to-end simulator speedup.

D. Transfer-Inclusive CUDA Benchmark

The CUDA experiment runs on a Google Colab Tesla T4 (compute capability 7.5) with PyTorch 2.11.0+cu128 and CUDA 12.8. Before accepting timings, CPU and GPU outputs are compared for numerical equivalence. Both GPU-resident and transfer-inclusive timings are recorded. The transfer-inclusive path explicitly includes CPU-to-GPU input movement, inference, synchronization, and result movement back to CPU.

At batch one, transfer-inclusive GPU execution is slower for every registered workload. For the temporal model at batch 256, GPU becomes advantageous with measured speedup

$$S_{256} = \frac{1281.69}{752.35} \approx 1.704 \times . \tag{33}$$

Maximum output-equivalence error is 0 for the imitation and PPO workloads and approximately 10^{-6} for the temporal model.

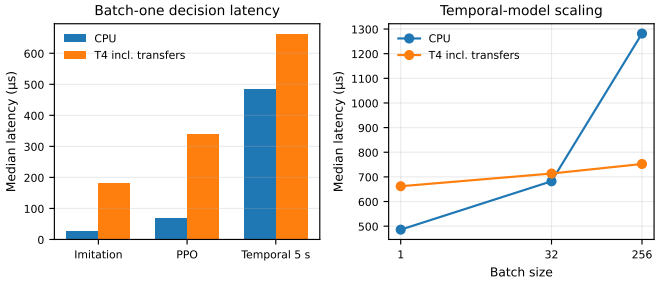


Fig. 9. CPU versus transfer-inclusive Tesla T4 inference. The latency-sensitive batch-one path favors CPU, while temporal inference becomes GPU-favorable at large batch.

XIV. DISCUSSION

A. Prediction Quality Is Not Decision Quality

The strongest recurring result is the non-monotonic mapping from probability metrics to execution decisions. A predictor can improve log loss without crossing an MPC decision boundary, while a less accurate or less calibrated signal can cross a boundary because its probability mass is distributed differently around the controller's effective threshold. The perfect-event-oracle experiment strengthens this interpretation: even exact knowledge of the intermediate quote-depletion target need not improve implementation shortfall. The economic objective depends on inventory, spread, queue uncertainty, urgency, terminal cost, and the alternative action sequence, not on the target alone.

This observation suggests that forecasting benchmarks should be reported alongside downstream action/value diagnostics whenever the forecast is consumed by an optimizer. It also motivates future decision-focused loss functions that weight errors by their local execution consequence rather than their standalone probability score.

B. Policy Complexity and Robustness

The deterministic liquidity-aware heuristic remains highly competitive. PPO produces good controlled ID mean cost and wins selected stress cells, but it does not dominate the simpler baselines and degrades under particular volatility/OOD regimes. This is consistent with the generalization concerns emphasized in recent RL execution work [14]. The result is not an argument against RL; rather, it shows that the burden of proof for an RL execution policy should include strong baselines, multiple seeds, OOD regimes, reward audits, and tail-risk metrics.

The imitation experiment provides a complementary lesson. A small student can exactly reproduce a teacher

on an in-distribution holdout and offer very low inference latency, yet still fail under shifted states. DAgger-style correction improves sequential agreement, while conservative fallback controls OOD tail cost at the price of invoking the expensive teacher frequently. In a practical system, fallback rate therefore becomes an explicit systems metric alongside action agreement.

C. Statistical Ranking Versus Point Ranking

The 43-cell robustness table is informative but insufficient on its own. More than half of the 129 paired confidence intervals cross zero, and 22 of 43 point winners fall below the 80% bootstrap ranking-stability diagnostic. This is a concrete example of why large strategy tables can create false confidence when each cell is interpreted independently. Paired resampling and family-wise multiplicity adjustment reveal which differences are robust enough to support stronger interpretation.

D. Latency as Part of the Decision Problem

The systems results reject the assumption that a GPU is automatically the correct inference platform for ML-assisted execution. For batch-one policies, the networks are so small that launch, synchronization, and host-device transfer dominate arithmetic. The T4 only becomes favorable for the temporal model at batch 256. The appropriate deployment architecture is therefore workload-dependent: CPU for latency-sensitive individual decisions; GPU for large batched temporal scoring or offline research workloads. Similar reasoning applies to compiler choices and language boundaries: an optimization is useful only after measuring the complete path that actually participates in the control loop.

E. Threats to Validity and Research Boundary

Construct validity: The quote-depletion target is an intermediate microstructure event, not execution utility. The oracle counterexample demonstrates this mismatch directly. Implementation shortfall and terminal completion are therefore computed from realized fills under common accounting, but they remain conditional on the registered benchmark and cost model.

Internal validity: Matched controllers, paired seeds, causal cutoffs, independent split roles, exact accounting, and zero-weight equivalence reduce confounding. Aggregate-L2 queue allocation, impact, and synthetic order flow are nevertheless scenario assumptions rather than estimated true-market parameters; the robustness matrix varies them but cannot eliminate misspecification.

Statistical validity: Each stress cell contains 24 paired episodes, PPO uses five registered seeds, and the moving-block bootstrap operates on synthetic episode seeds as pseudo-days. Confidence intervals and ranking probabilities quantify sampling uncertainty in this design, but they do not establish universal policy ordering or substitute for a larger historical panel.

External validity: Controlled state transitions isolate mechanisms but do not reproduce all hidden liquidity, exchange-specific priority rules, network jitter, outages, endogenous impact, or participant adaptation. Only one compact temporal architecture and one predeclared PPO family are evaluated; no architecture novelty or universal superiority is claimed. Any venue or historical regime is a new external-validity test.

Systems validity: CPU and CUDA results are hardware- and workload-specific process measurements, not exchange round-trip latency or high-frequency deployment guarantees. The transfer-inclusive boundary is measured, but production networking, colocated exchange gateways, and operational failover lie outside scope. These boundaries motivate a future confirmatory study: admit a qualifying multi-day historical dataset under the frozen Gate-C protocol and repeat the matched comparison without revising the hypothesis after observing outcomes.

XV. Reproducibility and Software Verification

The research system is built as staged, schema-validated software rather than a single notebook. Deterministic artifacts carry hashes and provenance identifiers; model weights use canonical serialization where practical; experiment configurations are versioned; and validators regenerate committed outputs byte-for-byte when scientific determinism is meaningful.

The native inventory contains 53 tests and reports 53/53 passes in each registered compiler or sanitizer matrix: GCC, Clang, AppleClang, optimized Release, ASan/UBSan, and ThreadSanitizer. Compiler warnings are errors. The Python inventory contains 481 tests with 91% combined branch-aware repository coverage. Independent semantic validators check data contracts, feature causality, model artifacts, controller equivalence, imitation/RL reports, robustness statistics, performance evidence, and the manuscript's headline claims. Expensive regeneration suites are partitioned rather than hidden behind a monolithic command timeout.

The final source tree additionally provides CMake packaging, an external consumer check, Docker/reproduction entry points, JSON Schemas for stage outputs, checksum manifests, a citation file, and a claim-to-artifact traceability table. The required-file and schema contracts and the release validator are automated in CI. Source, configurations, paper, figures, and evidence are publicly available at https://github.com/000zer000/robust-ml-optimal-execution; the paper can be rebuilt with one documented Tectonic target. This structure follows the general reproducibility principle that a computational result should be accompanied by sufficiently precise code, configuration, seeds, and compute information to make regeneration auditable [39].

XVI. Conclusion

This paper presented an integrated framework for studying robust machine-learning-assisted optimal execution

at the intersection of quantitative finance, market microstructure, machine learning, sequential control, and high-performance systems. The system begins with exact C++ event and matching semantics, models aggregate-L2 queue uncertainty explicitly, computes implementation shortfall from realized fills, and uses matched receding-horizon controllers to isolate the incremental effect of microstructure forecasts. It then extends the decision stack through calibrated temporal prediction, prediction-versus-decision ablations, imitation learning with DAgger, multi-seed PPO, a broad robustness matrix, paired block-bootstrap inference, and workload-specific CPU/GPU benchmarking.

Within the registered controlled-simulator evidence, the most important relationships are conditional rather than monotone. Better probability scores do not guarantee different or better execution actions; sophisticated learned policies do not uniformly dominate strong heuristics; strategy rankings change across stress regimes; point winners can be statistically fragile; and GPU acceleration becomes useful only when the workload is large enough to amortize transfer and launch overhead. These findings favor an execution-research methodology in which predictive models, controllers, robustness, statistics, and systems performance are evaluated through a common economic objective and a common causal accounting pipeline. Historical confirmation remains a future test, not an inference silently drawn from the controlled study.

Appendix A
Frozen Causal Feature Contract

Every feature obeys Equation (23). Feature construction is side-normalized so bid and ask decisions share the same semantic orientation where possible. Frozen naming and ordering prevent silent feature drift between model training and controller inference.

Appendix B
Temporal Learning Protocol

For each horizon, raw source rows are grouped by synthetic whole-day identifiers and transformed into eight-step sequences only within eligible chronological segments. The controlled split contains 100 simulator-generated day identifiers across BTCUSDT/ETHUSDT and bid/ask decisions. Sequence counts are 1,000 train, 400 validation, 200 calibration, and 400 holdout. Training-only normalization is applied featurewise. Validation chooses architecture/epoch; calibration fits the scalar Platt parameters; holdout predictions are emitted as immutable decision endpoints with provenance hashes.

The compact model was intentionally selected over a transformer architecture because the experiment aims to study downstream decision sensitivity without introducing an architecture sweep. The resulting parameter counts are approximately 1–2k, enabling batch-one CPU inference in the sub-millisecond range and making end-to-end compute-latency analysis meaningful.

Appendix C
ML-MPC Prediction Endpoint Contract

Every active ML-MPC decision consumes exactly one prediction record keyed by decision identifier. A valid record must satisfy:

1) prediction decision ID equals observation decision ID;
2) endpoint time equals decision time;
3) feature cutoff is no later than the observation cutoff;
4) prediction availability time is no later than decision time;
5) probability and base rate lie in $[0, 1]$;
6) horizon, model identifier, and provenance identifier are non-empty;
7) stale controls identify a strictly earlier same-group source endpoint;
8) duplicate active prediction identifiers are rejected.

Missing active predictions fail closed rather than reverting to an unrecorded default. This contract ensures that controller comparisons are not contaminated by inference-time bookkeeping differences.

Appendix D
Detailed PPO Configuration and Reward Audit

The reward audit stores both the decomposed reward and sufficient raw transition fields to recompute it. Let $\widetilde{C}_k$ be independently reconstructed cost from raw economics. The audit requires

$$\max_k |C_k - \widetilde{C}_k| \le \varepsilon, \tag{34}$$

with deterministic fixture result 0.0. The wait/no-op policy is separately evaluated so that a policy cannot obtain an attractive reward by avoiding execution until inventory disappears; terminal inventory is always completed and charged.

Appendix E
Robustness Dimension Registry

The registered robustness matrix spans the following families:

- latency and observation delay;
- decision-grid coarsening/refinement;
- thin/thick visible liquidity;
- spread compression/widening;
- volatility shocks;
- optimistic/central/pessimistic queue allocation;
- favorable/adverse/zero fee regimes;
- parent-size scaling relative to visible depth;
- short/long horizon proxies;
- impact coefficient and impact functional-form misspecification;
- calibrated, stale, shuffled, neutral, and degraded prediction inputs;
- information dropping and delayed observations;
- later-time and instrument-scale distribution shifts;
- compute-budget constraints;

TABLE VIII
Twenty Causal Microstructure Features

#	Feature	Unit / encoding	Information role
1	`spread_ticks`	ticks	instantaneous transaction-cost state
2	`same_top1_lots`	lots	passive-side top depth
3	`opposite_top1_lots`	lots	opposite-side top depth
4	`same_top5_lots`	lots	passive-side depth through level 5
5	`opposite_top5_lots`	lots	opposite-side depth through level 5
6	`side_imbalance_top1_bps`	scaled imbalance	side-aligned top-level pressure
7	`side_imbalance_top5_bps`	scaled imbalance	side-aligned multi-level pressure
8	`toward_quote_trade_flow_250ms_lots`	lots	recent toward-quote flow
9	`toward_quote_trade_flow_1s_lots`	lots	1-second toward-quote flow
10	`toward_quote_trade_flow_5s_lots`	lots	5-second toward-quote flow
11	`trade_count_1s`	count	recent activity intensity
12	`trade_count_5s`	count	medium-window activity intensity
13	`side_mid_move_250ms_half_ticks`	half-ticks	side-aligned short move
14	`side_mid_move_1s_half_ticks`	half-ticks	side-aligned 1-second move
15	`side_mid_move_5s_half_ticks`	half-ticks	side-aligned 5-second move
16	`realized_abs_mid_move_1s_half_ticks`	half-ticks	local volatility proxy
17	`realized_abs_mid_move_5s_half_ticks`	half-ticks	medium-window volatility proxy
18	`spread_change_1s_ticks`	ticks	spread dynamics
19	`quote_age_ns`	ns	quote staleness
20	`time_since_last_trade_ns`	ns	event-time staleness

TABLE IX
Registered PPO Configuration

Parameter	Value
Policy hidden layers	32, 32 (tanh)
Discount γ	0.99
GAE λ	0.95
Clip ϵ	0.20
Entropy coefficient	0.01
Value coefficient	0.50
Learning rate	0.001
Train episodes/update	20
Epochs/update	4
Updates	10
Registered seeds	27, 127, 227, 327, 427

- favorable/adverse simulator mismatch and combined unseen regimes.

The same paired episode seeds are reused within comparable cells so policy differences are not inflated by independent environment draws.

Appendix F
Statistical Procedure

For each challenger and comparator within a stress family, the analysis computes paired mean difference, median difference, standardized effect diagnostics, and block-bootstrap confidence intervals. Holm adjustment is applied within registered exploratory families. Ranking stability is obtained by jointly resampling the common episode index and recomputing all policy means in each bootstrap replication. This preserves cross-policy dependence and provides an interpretable quantity:

$$\widehat{\Pr}(A = \arg\min_j \bar{C}_j^*) = \frac{1}{B}\sum_{b=1}^{B} \mathbf{1}\{A = \arg\min_j \bar{C}_{j,b}^*\}. \quad (35)$$

The procedure is designed to distinguish a point-estimate winner from a ranking that remains stable under paired sampling variation.

Appendix G
Systems and Verification Matrix

The native tests are compiled under independent compiler/configuration matrices. Sanitizer coverage detects memory/undefined-behavior failures. Release installation is additionally checked through an external CMake consumer using the public target. Performance artifacts are kept distinct from correctness artifacts because raw timing is hardware-specific while semantic checks must remain deterministic.

Appendix H
Software Architecture and Artifact Interfaces

The implementation separates correctness-critical market mechanics from research orchestration. C++ owns exact event types, price/quantity arithmetic, matching, replay primitives, action validation, and MPC search. Python owns feature construction, supervised learning, temporal models, imitation, RL, robustness generation, statistics, and benchmark orchestration. Cross-stage artifacts are schema-validated JSON/CSV rather than implicit in-memory notebook state.

Artifact schemas are versioned and validators check required fields, units, causal timestamps, provenance identifiers, and expected source hashes. This architecture makes model and controller stages replaceable without weakening the market-state or accounting contracts.

Appendix I
Additional Systems Measurements

The matching benchmark uses 25 raw repetitions per baseline/optimized thread count and preserves each timing vector in the performance artifact. CUDA comparisons use ten measured repetitions per model/batch after warm-up, report median, mean, median absolute deviation, p95, minimum, maximum, and raw samples, and verify output

TABLE X
Final Verification Matrix at the Systems Boundary

Layer	Verification mechanism	Final registered result
C++ native	GCC / Clang / AppleClang / Release / ASan+UBSan / TSan	53/53 each
Python	partitioned unit/integration suites	481 tests
Coverage	branch-aware combined repository measurement	91%
Specification	frozen-file hash lock	4/4
Repository contract	required-file/schema contract	automated pass
Prediction	leakage/mutation/provenance validators	pass
ML-MPC	shared-solver and zero-weight equivalence	pass
Temporal model	deterministic model/prediction artifacts	pass
Imitation	exact teacher-data/policy regeneration	pass
RL	multi-seed policy/report regeneration and reward audit	pass
Robustness	registered stress matrix regeneration	pass
Statistics	contrast/ranking regeneration	pass
Performance	correctness after optimization + hardware evidence	pass

TABLE XI
Major Software Layers

Layer	Primary language	Responsibility	Representative verification
Market core	C++	exact ticks/lots, event ordering, matching, order lifecycle	native unit tests, invariants, sanitizers
Replay/queue	C++ / Python	historical replay contracts, aggregate-L2 queue scenarios	deterministic fixtures, queue assumption tests
Execution accounting	C++ / Python	fills, fees, benchmark notional, implementation shortfall	independent accounting checks
MPC	C++	bounded action-tree search, terminal completion, ML risk input	shared-solver equivalence tests
Prediction	Python	20-feature causal contract, models, calibration, endpoint tapes	mutation/leakage tests, artifact hashes
Imitation	Python + C++ oracle	behavior cloning, DAgger correction, OOD fallback	exact teacher labels, policy reconstruction
RL	Python / PyTorch	interactive MDP, PPO, reward accounting	multi-seed regeneration, reward reconstruction
Robustness and stats	Python	stress matrix, paired bootstrap, Holm, ranking stability	byte-identical report regeneration
Performance	C++ / Python / CUDA	profiling, threading, compiled inference, boundary and GPU timing	output equivalence + correctness reruns

equivalence before timing is accepted. These raw distributions are retained because microsecond-scale timings on shared/virtualized hardware can be noisy.

For batch-one CUDA measurements, the transfer-inclusive GPU/CPU median ratios are approximately 6.34 for imitation, 4.88 for PPO, and 1.36 for the temporal model (ratios greater than one indicate slower GPU execution). At batch 256 the temporal ratio reverses to approximately 0.587, corresponding to the 1.704× CPU/GPU speedup reported in Table VII. The crossover is therefore a batching effect rather than a statement about GPU arithmetic in isolation.

Appendix J
Notation Summary

TABLE XII
Principal Symbols

Symbol	Meaning
Q	parent-order quantity
I_k	remaining inventory at decision k
s	side sign, $+1$ buy / -1 sell
P_0	arrival benchmark price
P_i, q_i	fill price and quantity
F	total explicit fees/rebates
x_k	causal controller state
a_k	child action
p	predicted depletion/trade-through probability
$\bar{p}$	training base rate
λ_p	prediction-risk sensitivity
λ_I	inventory-risk coefficient
λ_T, λ_Q	terminal penalties

References

[1] D. Bertsimas and A. W. Lo, "Optimal control of execution costs," *Journal of Financial Markets*, vol. 1, no. 1, pp. 1–50, 1998.

[2] R. Almgren and N. Chriss, "Optimal execution of portfolio transactions," *The Journal of Risk*, vol. 3, no. 2, pp. 5–39, 2001.

[3] A. A. Obizhaeva and J. Wang, "Optimal trading strategy and supply/demand dynamics," *Journal of Financial Markets*, vol. 16, no. 1, pp. 1–32, 2013.

[4] A. Alfonsi, A. Fruth, and A. Schied, "Optimal execution strategies in limit order books with general shape functions," *Quantitative Finance*, vol. 10, no. 2, pp. 143–157, 2010.

[5] J. Gatheral, "No-dynamic-arbitrage and market impact," *Quantitative Finance*, vol. 10, no. 7, pp. 749–759, 2010.

[6] Z. Zhang, S. Zohren, and S. Roberts, "DeepLOB: Deep convolutional neural networks for limit order books," *IEEE Transactions on Signal Processing*, vol. 67, no. 11, pp. 3001–3012, 2019.

[7] J. Sirignano and R. Cont, "Universal features of price formation in financial markets: Perspectives from deep learning," *Quantitative Finance*, vol. 19, no. 9, pp. 1449–1459, 2019.

[8] P. N. Kolm, J. Turiel, and N. Westray, "Deep order flow imbalance: Extracting alpha at multiple horizons from the limit order book," *Mathematical Finance*, vol. 33, no. 4, pp. 1044–1081, 2023.

[9] A. Briola, S. Bartolucci, and T. Aste, "Deep limit order book forecasting: A microstructural guide," *Quantitative Finance*, vol. 25, no. 7, pp. 1101–1131, 2025.

[10] A. N. Elmachtoub and P. Grigas, "Smart "predict, then optimize"," *Management Science*, vol. 68, no. 1, pp. 9–26, 2022.

[11] P. Donti, B. Amos, and J. Z. Kolter, "Task-based end-to-end model learning in stochastic optimization," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[12] B. Wilder, B. Dilkina, and M. Tambe, "Melding the data-decisions pipeline: Decision-focused learning for combinatorial optimization," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, 2019, pp. 1658–1665.

[13] Y. Nevmyvaka, Y. Feng, and M. Kearns, "Reinforcement learning for optimized trade execution," in *Proceedings of the 23rd International Conference on Machine Learning*, 2006, pp. 673–680.

[14] C. Zhang, Y. Duan, X. Chen, J. Chen, J. Li, and L. Zhao, "Towards generalizable reinforcement learning for trade execution," in *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*, 2023, pp. 4975–4983.

[15] B. Wang, X. Gao, and L. Li, "Reinforcement learning for continuous-time optimal execution: Actor–critic algorithm and error analysis," *Finance and Stochastics*, vol. 30, pp. 597–655, 2026.

[16] P. Cheridito and M. Weiss, "Reinforcement learning for trade execution with market and limit orders," *Quantitative Finance*, 2026.

[17] J. Lorenz and R. Almgren, "Mean–variance optimal adaptive execution," *Applied Mathematical Finance*, vol. 18, no. 5, pp. 395–422, 2011.

[18] Á. Cartea and S. Jaimungal, "Optimal execution with limit and market orders," *Quantitative Finance*, vol. 15, no. 8, pp. 1279–1291, 2015.

[19] O. Guéant, C.-A. Lehalle, and J. Fernandez-Tapia, "Optimal portfolio liquidation with limit orders," *SIAM Journal on Financial Mathematics*, vol. 3, no. 1, pp. 740–764, 2012.

[20] R. Cont, S. Stoikov, and R. Talreja, "A stochastic model for order book dynamics," *Operations Research*, vol. 58, no. 3, pp. 549–563, 2010.

[21] R. Cont and A. de Larrard, "Price dynamics in a markovian limit order market," *SIAM Journal on Financial Mathematics*, vol. 4, no. 1, pp. 1–25, 2013.

[22] W. Huang, C.-A. Lehalle, and M. Rosenbaum, "Simulating and analyzing order book data: The queue-reactive model," *Journal of the American Statistical Association*, vol. 110, no. 509, pp. 107–122, 2015.

[23] R. Cont, A. Kukanov, and S. Stoikov, "The price impact of order book events," *Journal of Financial Econometrics*, vol. 12, no. 1, pp. 47–88, 2014.

[24] J.-P. Bouchaud, J. D. Farmer, and F. Lillo, "How markets slowly digest changes in supply and demand," in *Handbook of Financial Markets: Dynamics and Evolution*. North-Holland, 2009, pp. 57–160.

[25] M. D. Gould, M. A. Porter, S. Williams, M. McDonald, D. J. Fenn, and S. D. Howison, "Limit order books," *Quantitative Finance*, vol. 13, no. 11, pp. 1709–1742, 2013.

[26] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 70, 2017, pp. 1321–1330.

[27] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 15, 2011, pp. 627–635.

[28] T. España, Y. Hafsi, F. Lillo, and E. Vittori, "Reinforcement learning in queue-reactive models: Application to optimal execution," 2025, preprint.

[29] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017.

[30] D. Byrd, M. Hybinette, and T. H. Balch, "ABIDES: Towards high-fidelity multi-agent market simulation," in *Proceedings of the 2020 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation*, 2020, pp. 11–22.

[31] S. Frey, K. Li, P. Nagy, S. Sapora, C. Lu, S. Zohren, J. Foerster, and A. Calinescu, "JAX-LOB: A gpu-accelerated limit order book simulator to unlock large-scale reinforcement learning for trading," in *Proceedings of the Fourth ACM International Conference on AI in Finance*, 2023, pp. 583–591.

[32] S. Vyetrenko, D. Byrd, N. Petosa, M. Mahfouz, D. Dervovic, M. Veloso, and T. H. Balch, "Get real: Realism metrics for robust limit order book market simulations," in *Proceedings of the First ACM International Conference on AI in Finance*, 2020.

[33] P. Nagy, S. Y. Frey, K. Li, B. Sarkar, S. Vyetrenko, S. Zohren, A. Calinescu, and J. N. Foerster, "LOB-bench: Benchmarking generative ai for finance—an application to limit order book data," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 45 437–45 460.

[34] H. Bodor and L. Carlier, "Deep learning meets queue-reactive: A framework for realistic limit order book simulation," 2025, preprint.

[35] P. Noble, M. Rosenbaum, and S. Souilmi, "Bridging the reality gap in limit order book simulation," 2026, preprint.

[36] D. N. Politis and J. P. Romano, "The stationary bootstrap," *Journal of the American Statistical Association*, vol. 89, no. 428, pp. 1303–1313, 1994.

[37] H. White, "A reality check for data snooping," *Econometrica*, vol. 68, no. 5, pp. 1097–1126, 2000.

[38] J. P. Romano and M. Wolf, "Stepwise multiple testing as formalized data snooping," *Econometrica*, vol. 73, no. 4, pp. 1237–1282, 2005.

[39] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Lariviere, A. Beygelzimer, F. d'Alche Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.